

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Цатинян Артём

Группа К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Тема лабораторного проекта: приложение для бронирования столиков в ресторанах.

В рамках лабораторной работы требовалось реализовать полностью работающее REST API на основе выбранного boilerplate с учетом ранее выполненных домашних заданий по проектированию базы данных и API. Необходимо было реализовать модели, представления и контроллеры, а также продемонстрировать работоспособность API согласно выбранному варианту.

Ход работы

Для выполнения лабораторной работы был реализован backend-сервис на Kotlin с использованием Spring Boot 3. В качестве системы управления базами данных использовалась PostgreSQL. Для управления структурой базы данных были подготовлены миграции Liquibase в формате XML. Для работы с базой данных и генерации типов применялся jOOQ, а для преобразования DTO и внутренних моделей использовался MapStruct.

Структура проекта была организована по сущностям предметной области. Для каждой сущности были выделены слои adapter, service и domain. В слое adapter.jdbc размещены DAO-классы, наследующиеся от сгенерированных jOOQ DAO. В слое adapter.rest размещены контроллеры и DTO для запросов и ответов. Такой подход позволил разделить HTTP-слой, бизнес-логику и доступ к данным.

В базе данных была реализована схема для предметной области бронирования столиков в ресторанах. Были описаны и связаны между собой пользователи, рестораны, кухни, рабочие часы ресторанов, столики, фотографии, категории меню, позиции меню, бронирования и отзывы. Все основные идентификаторы сущностей реализованы как bigint с автоинкрементом. Для локальной разработки использовалась база данных storage_local и схема storage.

В рамках реализации API были разработаны следующие основные группы эндпоинтов:

- аутентификация и авторизация: POST /api/v1/auth/register, POST /api/v1/auth/login;
- профиль пользователя: GET /api/v1/users/me, PATCH /api/v1/users/me;
- справочник кухонь: GET /api/v1/cuisines;
- работа с ресторанами: GET /api/v1/restaurants, GET /api/v1/restaurants/{restaurantId}, GET /api/v1/restaurants/{restaurantId}/availability, GET /api/v1/restaurants/{restaurantId}/menu, GET /api/v1/restaurants/{restaurantId}/reviews;
- работа с бронированиями: POST /api/v1/bookings, GET /api/v1/bookings/me, GET /api/v1/bookings/{bookingId}, PATCH /api/v1/bookings/{bookingId}/cancel;
- работа с отзывами: POST /api/v1/restaurants/{restaurantId}/reviews.

При проектировании и реализации API были учтены результаты Д32. Для запросов были подготовлены DTO с валидацией входных данных. Для защищенных методов использовалась JWT-аутентификация. Также была подключена Swagger/OpenAPI-документация для удобной ручной проверки эндпоинтов.

Дополнительно для проекта был подготовлен SQL-скрипт с тестовыми данными. В систему были добавлены несколько ресторанов, кухни, меню, фотографии, бронирования и отзывы, что позволило проверять API на реалистичном наборе данных. Это упростило ручное тестирование сценариев поиска ресторанов, просмотра меню, создания бронирований и публикации отзывов.

После реализации был выполнен smoke-test приложения. Были проверены следующие пользовательские сценарии:

- регистрация нового пользователя;
- вход в систему по email и паролю;
- получение данных текущего пользователя;
- получение списка кухонь;
- поиск ресторанов с фильтрацией;
- просмотр детальной информации о ресторане;
- просмотр меню и доступных слотов бронирования;

- создание бронирования;
- просмотр собственных бронирований;
- отмена созданного бронирования;
- создание отзыва по завершеному бронированию.

В ходе тестирования приложение успешно запускалось, миграции Liquibase применялись корректно, API возвращало ожидаемые ответы, а Swagger UI позволял выполнять проверку ручек в интерактивном режиме.

Вывод

В результате выполнения лабораторной работы было реализовано работающее REST API для приложения бронирования столиков в ресторанах. Были подготовлены структура проекта, схема базы данных, миграции Liquibase, DAO на основе jOOQ, сервисы, контроллеры и DTO. Реализованный backend покрывает основные пользовательские сценарии системы: регистрацию, авторизацию, поиск ресторанов, просмотр меню, создание бронирований и публикацию отзывов. Полученный результат соответствует поставленной задаче и может использоваться как основа для дальнейшего развития проекта.